

5.3 1.9.3 Metoda bazată pe principiul rezoluției pentru verificarea validității formulelor limbajului calculului cu propoziții.

Rezoluția a fost introdusă ca regulă de inferență de către J.A.Robinson în 1965 și constituie sistemul deductiv cel mai frecvent utilizat în demonstrarea automată.

Reamintim că rezoluția este o regulă de inferență derivată, reprezentată convențional prin $\frac{(\alpha \rightarrow \beta), ((\neg \alpha) \rightarrow \gamma)}{(\beta \vee \gamma)} REZ$ (Aplicația 1.4.9).

Observație Deoarece $((\alpha \rightarrow \beta) \rightarrow (((\neg \alpha) \rightarrow \gamma) \rightarrow (\beta \vee \gamma))) \in T_h$, aplicând teorema deducției obținem $\{(\alpha \rightarrow \beta), ((\neg \alpha) \rightarrow \gamma)\} \vdash (\beta \vee \gamma)$ sau echivalent $\{(\alpha \rightarrow \beta), ((\neg \alpha) \rightarrow \gamma)\} \vdash \{(\beta \vee \gamma)\}$ și care din punct de vedere semantic revine la $\{(\alpha \rightarrow \beta), ((\neg \alpha) \rightarrow \gamma)\} \models \{(\beta \vee \gamma)\}$,

adică $\aleph((\alpha \rightarrow \beta)) \cap \aleph(((\neg \alpha) \rightarrow \gamma)) \subset \aleph((\beta \vee \gamma))$.

Deoarece $\aleph((\alpha \rightarrow \beta)) = (\mathfrak{S} \setminus \aleph(\alpha)) \cup \aleph(\beta) = \aleph((\neg \alpha)) \cup \aleph(\beta) =$

$\aleph(((\neg \alpha) \vee \beta))$ și

$\aleph(((\neg \alpha) \rightarrow \gamma)) = (\mathfrak{S} \setminus \aleph((\neg \alpha))) \cup \aleph(\gamma) = \aleph(\alpha) \cup \aleph(\gamma) = \aleph((\alpha \vee \gamma))$

obținem,

$\aleph(((\neg \alpha) \vee \beta)) \cap \aleph((\alpha \vee \gamma)) \subset \aleph((\beta \vee \gamma))$,

adică $\{((\neg \alpha) \vee \beta), (\alpha \vee \gamma)\} \models \{(\beta \vee \gamma)\}$,

ceea ce este echivalent cu $\{((\neg \alpha) \vee \beta), (\alpha \vee \gamma)\} \vdash (\beta \vee \gamma)$.

Definiția 1.9.3.1 Fie k_1, k_2 clauze și λ literal. Clauzele k_1, k_2 sunt λ -rezolubile, dacă $k_1 \langle \lambda \rangle$ și $k_2 \langle (\neg \lambda) \rangle$. Clauza $rez_\lambda(k_1, k_2) = (k_1 \setminus \lambda) \vee (k_2 \setminus (\neg \lambda))$ este

λ -rezolventa perechii de clauze (k_1, k_2) . Clauzele k_1, k_2 se numesc clauze parentale ale rezolventei.

Observație În particular, pentru $\alpha = \lambda, \beta = (k_2 \setminus (\neg \lambda))$,

$\gamma = (k_1 \setminus \lambda)$, din observația precedentă rezultă $\{k_1, k_2\} \models rez_\lambda(k_1, k_2)$ sau echivalent, $\aleph(k_1) \cap \aleph(k_2) \subset \aleph(rez_\lambda(k_1, k_2))$. Evident, dacă nici una din clauzele k_1, k_2 nu este tautologie, atunci $rez_\lambda(k_1, k_2) \langle \lambda \rangle$ și $rez_\lambda(k_1, k_2) \langle (\neg \lambda) \rangle$.

Exemplul 1.9.3.1 Fie $k_1 = a \vee (\neg b) \vee c, k_2 = (\neg a) \vee (\neg c)$,

$k_3 = (\neg b) \vee (\neg a)$.

Clauzele k_1, k_2 sunt a -rezolubile și c -rezolubile.

$rez_a(k_1, k_2) = (\neg b) \vee c \vee (\neg c)$

$rez_c(k_1, k_2) = a \vee (\neg b) \vee (\neg a)$

Clauzele k_1, k_3 sunt a -rezolubile

$$\text{rez}_a(k_1, k_3) = (\neg b) \vee c \vee (\neg b) \equiv c \vee (\neg b)$$

Clauzele k_2, k_3 nu sunt rezolubile

Din exemplele considerate rezultă că este posibil ca rezolventa a două clauze să fie clauză tautologie în condițiile în care ambele clauze parentale nu sunt tautologii. De asemenea, este posibil ca în rezolventa unei perechi de clauze să fie generate duplicate ale unuia sau mai mulți literal.

Definiția 1.9.3.2 Fie $S(\alpha)$ o reprezentare clauzală liberă de tautologii și λ literal. Rezoluția $REZ_\lambda(\alpha)$ în raport cu literalul λ este reprezentarea clauzală,

$$REZ_\lambda(\alpha) = \alpha_\lambda^0 \cup \{ \text{rez}_\lambda(k_1, k_2) \mid k_1 \in \alpha_\lambda^+, k_2 \in \alpha_\lambda^- \}$$

Exemplul 1.9.3.2 Dacă

$$S(\alpha) = \{ (\neg p) \vee o, (\neg p) \vee (\neg c), (\neg m) \vee c \vee i, m, p, (\neg i) \},$$

atunci,

$$REZ_p(\alpha) = \{ (\neg m) \vee c \vee i, m, (\neg i), o, (\neg c) \}$$

$$REZ_c(REZ_p(\alpha)) = \{ (\neg m) \vee i, m, (\neg i), o \}$$

$$REZ_i(REZ_c(REZ_p(\alpha))) = \{ (\neg m), m, o \}$$

$$REZ_m(REZ_i(REZ_c(REZ_p(\alpha)))) = \{ \square, o \}.$$

Teorema 1.9.3.1 (Teorema de bază a rezoluției). Fie $S(\alpha)$ o reprezentare clauzală.

Dacă $S(\alpha)$ este validabilă, atunci pentru orice literal λ , $REZ_\lambda(\alpha)$ este validabilă. Dacă există un literal λ , astfel încât $REZ_\lambda(\alpha)$ este validabilă, atunci $S(\alpha)$ este validabilă.

Demonstrație Presupunem că $S(\alpha)$ este validabilă și fie λ literal arbitrar. Fie $I \in \aleph(S(\alpha))$ arbitrară, deci $I \in \bigcap_{k \in S(\alpha)} \aleph(k)$.

Pentru orice $k \in REZ_\lambda(\alpha)$ obținem

a) dacă $k \in \alpha_\lambda^0$, deoarece $\alpha_\lambda^0 \subset S(\alpha)$ rezultă $I(k) = T$;

b) dacă $k = \text{rez}_\lambda(k_1, k_2)$ pentru anume $k_1 \in \alpha_\lambda^+, k_2 \in \alpha_\lambda^-$, atunci, cum $\{k_1, k_2\} \models \text{rez}_\lambda(k_1, k_2)$, rezultă $I(k) = T$.

Obținem astfel $I \in \bigcap_{k \in REZ_\lambda(\alpha)} \aleph(k) = \aleph(REZ_\lambda(\alpha))$, deci $REZ_\lambda(\alpha)$ este

validabilă. În particular, deoarece pentru orice $I \in \aleph(S(\alpha))$ a rezultat

$$I \in \aleph(REZ_\lambda(\alpha)), \text{ obținem } \aleph(S(\alpha)) \subset \aleph(REZ_\lambda(\alpha)).$$

Presupunem că există λ literal, astfel încât $REZ_\lambda(\alpha)$ este validabilă și fie $I \in \aleph(REZ_\lambda(\alpha))$ arbitrară.

Dacă $I \in \aleph(POS_\lambda(\alpha))$, atunci, utilizând rezultatul stabilit de

Teorema 1.9.2.1, rezultă $S(\alpha)$ validabilă. Dacă $I \notin \aleph(POS_\lambda(\alpha))$, atunci există $k \in POS_\lambda(\alpha)$, astfel încât $I(k) = F$. În acest caz, deoarece

$$\alpha_\lambda^0 \subset REZ_\lambda(\alpha), \text{ obținem } k = k_1 \setminus \lambda \text{ pentru anume } k_1 \in \alpha_\lambda^+.$$

Fie $k^* \in NEG_\lambda(\alpha)$ arbitrară.

a) dacă $k^* \in \alpha_\lambda^0 \subset REZ_\lambda(\alpha)$, atunci $I(k^*) = T$;
b) dacă $k^* = k_2 \setminus (\neg \lambda)$ pentru anume $k_2 \in \alpha_\lambda^-$, atunci k_1, k_2 sunt λ -rezolubile
și $rez_\lambda(k_1, k_2) \in REZ_\lambda(\alpha)$ deci $I(rez_\lambda(k_1, k_2)) = T$.

Obținem

$$\begin{aligned} I(rez_\lambda(k_1, k_2)) &= I((k_1 \setminus \lambda) \vee (k_2 \setminus (\neg \lambda))) = \\ &= I((k_1 \setminus \lambda)) \vee I((k_2 \setminus (\neg \lambda))) = I(k) \vee I(k^*) = F \vee I(k^*) = I(k^*), \end{aligned}$$

deci $I(k^*) = T$.

Rezultă $I \in \bigcap_{k^* \in NEG_\lambda(\alpha)} \aleph(k^*) = \aleph(NEG_\lambda(\alpha))$, deci $NEG_\lambda(\alpha)$ este validabilă. Utilizând rezultatul stabilit de *Teorema 1.9.2.1*, rezultă $S(\alpha)$ validabilă.

În particular, a rezultat că pentru orice $I \in \aleph(REZ_\lambda(\alpha))$, $I \in \aleph(POS_\lambda(\alpha))$ sau $I \in \aleph(NEG_\lambda(\alpha))$, deci

$$\aleph(REZ_\lambda(\alpha)) \subset \aleph(POS_\lambda(\alpha)) \cup \aleph(NEG_\lambda(\alpha)).$$

Corolarul 1.9.3.1 Dacă $S(\alpha)$ este o reprezentare clauzală, atunci

$$\begin{aligned} \aleph(S(\alpha)) &\subset \bigcap_{\lambda \in V \cup \neg V} \aleph(REZ_\lambda(\alpha)) \subset \\ &\subset \bigcap_{\lambda \in V \cup \neg V} (\aleph(NEG_\lambda(\alpha)) \cup \aleph(POS_\lambda(\alpha))). \end{aligned}$$

Demonstrație Din argumentele utilizate în demonstrația *Teoremei 1.9.3.1* a rezultat că pentru orice λ literal,

$$\aleph(S(\alpha)) \subset \aleph(REZ_\lambda(\alpha)), \text{ deci } \aleph(S(\alpha)) \subset \bigcap_{\lambda \in V \cup \neg V} \aleph(REZ_\lambda(\alpha)).$$

De asemenea, pentru fiecare λ literal, a rezultat relația

$$\aleph(REZ_\lambda(\alpha)) \subset \aleph(POS_\lambda(\alpha)) \cup \aleph(NEG_\lambda(\alpha)), \text{ deci}$$

$$\bigcap_{\lambda \in V \cup \neg V} \aleph(REZ_\lambda(\alpha)) \subset \bigcap_{\lambda \in V \cup \neg V} (\aleph(NEG_\lambda(\alpha)) \cup \aleph(POS_\lambda(\alpha))).$$

Definiția 1.9.3.3 Fie $S(\alpha)$ o reprezentare clauzală liberă de tautologii.

Secvența de clauze $k_1, \dots, k_n$ este o $S(\alpha)$ -derivare rezolutivă dacă pentru orice i , $1 \leq i \leq n$, este îndeplinită una din condițiile:

ι) $k_i \in S(\alpha)$;

$\iota\iota$) există $1 \leq j, p \leq i$ și există λ literal, astfel încât $k_i = rez_\lambda(k_j, k_p)$.

$S(\alpha)$ -derivarea rezolutivă $k_1, \dots, k_n$ este o $S(\alpha)$ -respingere rezolutivă, dacă $k_n = \square$.

Teorema 1.9.3.2 (Teorema de completitudine a rezoluției). Fie $S(\alpha)$ o reprezentare clauzală. Atunci $S(\alpha)$ este invalidabilă, dacă și numai dacă

există o $S(\alpha)$ – respingere rezolutivă.

Demonstrație Demonstrăm că pentru orice $S(\alpha)$ – derivare rezolutivă

$k_1, \dots, k_n, S(\alpha) \models k_i, 1 \leq i \leq n$.

Evident, $k_1 \in S(\alpha)$, deci cum $\aleph(S(\alpha)) = \bigcap_{k \in S(\alpha)} \aleph(k) \subset \aleph(k_1)$, rezultă

$S(\alpha) \models k_1$.

Presupunem $S(\alpha) \models k_i, \forall i, 1 \leq i \leq j \leq n-1$.

Dacă $k_{j+1} \in S(\alpha)$, atunci din $\aleph(S(\alpha)) = \bigcap_{k \in S(\alpha)} \aleph(k) \subset \aleph(k_{j+1})$ obținem

$S(\alpha) \models k_1$.

Dacă există $i, p, 1 \leq i, p \leq j$ și λ literal, astfel încât

$k_{j+1} = rez_\lambda(k_i, k_p)$, utilizând ipoteza inductivă, obținem

$\aleph(S(\alpha)) \subset \aleph(k_i) \cap \aleph(k_p) \subset \aleph(rez_\lambda(k_i, k_p))$ deci $S(\alpha) \models k_{j+1}$.

Presupunem că există o $S(\alpha)$ – derivare rezolutivă $k_1, \dots, k_n = \square$.

Din $\aleph(S(\alpha)) \subset \aleph(\square) = \emptyset$ obținem evident $\aleph(S(\alpha)) = \emptyset$, deci $S(\alpha)$ este invalidabilă.

Reciproc, presupunem că $S(\alpha)$ este invalidabilă. Fără restrângerea generalității, putem presupune că $S(\alpha)$ este liberă de tautologii.

Fie $\lambda_1, \dots, \lambda_n$ literalii care au ocurențe în clauzele mulțimii $S(\alpha)$. Considerăm secvența de reprezentări clauzale $S_0(\alpha) = S(\alpha)$,

$S_i(\alpha) = REZ_{\lambda_i}(S_{i-1}(\alpha)), i = 1, \dots, n$.

Presupunem de asemenea că $S_i(\alpha), i = 1, \dots, n$ sunt libere de tautologii.

Rezultă că pentru fiecare $i = 1, \dots, n$, literalii $\lambda_1, \dots, \lambda_i$ nu au ocurențe în clauzele mulțimii $S_i(\alpha)$, deci $S_n(\alpha) = \emptyset$ sau

$S_n(\alpha) = \{\square\}$.

Dacă $S_n(\alpha) = \emptyset$, atunci $S_{n-1}(\alpha)$ este validabilă.

Din *Teorema* 1.9.3.1 rezultă $S_i(\alpha)$ validabilă, $i = 0, \dots, n$, deci $S(\alpha)$ este validabilă.

Analog, dacă $S_n(\alpha) = \{\square\}$, atunci $S_{n-1}(\alpha)$ este invalidabilă, deci iterând același argument, obținem $S_i(\alpha)$ invalidabilă, $i = 0, \dots, n$.

În ipoteza că $S(\alpha)$ este invalidabilă rezultă $S_n(\alpha) = \{\square\}$.

Dacă $S_i(\alpha) = \{\gamma_i^1, \dots, \gamma_i^{m_i}\}, i = 0, \dots, n$, unde $m_n = 1$ și $\gamma_n^1 = \square$, atunci $\gamma_1^1, \dots, \gamma_1^{m_1}, \gamma_2^1, \dots, \gamma_2^{m_2}, \dots, \gamma_{n-1}^1, \dots, \gamma_{n-1}^{m_{n-1}}, \gamma_n^1$ este o

$S(\alpha)$ – derivare rezolutivă.

Deoarece $\gamma_n^1 = \square$, obținem în final că

$\gamma_1^1, \dots, \gamma_1^{m_1}, \gamma_2^1, \dots, \gamma_2^{m_2}, \dots, \gamma_{n-1}^1, \dots, \gamma_{n-1}^{m_{n-1}}, \gamma_n^1 = \square$

este o $S(\alpha)$ – respingere rezolutivă.

Rezultatul stabilit de *Teorema* 1.9.3.2 constituie suportul unui sistem de demonstrare automată pentru verificarea validabilității reprezentărilor clauzale. În esență, metoda revine la tentativa de obținere a unei respingeri rezolutive prin substituirea reprezentării clauzale curente printr-o reprezentare

clauzală cu număr de literal strict mai mic, dar posibil cu mai multe clauze. Deoarece prin luarea rezolventelor este posibilă generarea de clauze tautologii, în scopul menținerii reprezentărilor clauzale libere de tautologii este necesar ca la fiecare etapă să fie aplicat un test pentru detectarea și eliminarea rezolventelor tautologii nou generate. Spre deosebire de tehnica Davis-Putnam, implementarea demonstratorului rezolutiv nu necesită menținerea unei structuri de date suplimentare. Similar metodei Davis-Putnam, aplicarea priorității a regulilor literalilor puri și, respectiv, a clauzei unitare este o modalitate de prevenire a creșterii excesive a numărului de clauze.

Descrierea algoritmului de demonstrare automată rezolutiv este:

procedure DemRez;

Intrare: Structură de date pentru stocarea reprezentării clauzale $S(\alpha)$;

EliminaTautologii($S(\alpha)$);

EliminaDuplicate($S(\alpha)$);

$\gamma \leftarrow S(\alpha)$; $sw \leftarrow false$;

repeat

if $\gamma = \emptyset$ then

write ('validabilă');

$sw \leftarrow true$

else

if $\square \in \gamma$ then

write ('invalidabilă');

$sw \leftarrow true$

else

if (există λ clauza unitară) or

(există λ literal pur)

then

$\gamma \leftarrow NEG_{\lambda}(\gamma)$

else

alege(λ literal);

$\gamma \leftarrow RES_{\lambda}(\gamma)$;

EliminaTautologii(γ);

EliminaDuplicate(γ);

endif

endif

endif

until sw

end

Observație Similar algoritmului DvP, apelul EliminaTautologii(γ) detectează și elimină clauzele tautologii din reprezentarea clauzală curentă γ

(inițial $\gamma = S(\alpha)$). Apelul este necesar la fiecare etapă în care noua reprezentare clauzală este calculată ca rezoluție a reprezentării clauzale curente. Apelul EliminaDuplicate(γ) detectează și elimină duplicatele literalilor din fiecare clauză a reprezentării clauzale curente. Apelul alege(λ literal) efectuează selectarea unui literal pentru aplicarea rezoluției.

Exemplul 1.9.3.3 Procedura DemRez aplicată reprezentării clauzale

$$S(\alpha) = \{(\neg a) \vee (\neg c) \vee b, (\neg b) \vee c, (\neg c) \vee a, a \vee (\neg b) \vee (\neg c)\},$$

determină următoarea evoluție:

Inițializare: $\gamma \leftarrow S(\alpha)$, $sw \leftarrow false$.

Iterația 1: Nu există clauze unitare și nici literal puri.

Apelul alege(λ literal) $\Rightarrow \lambda = a$

$$\gamma \leftarrow REZ_a(\gamma) = \{(\neg b) \vee c, (\neg c) \vee b \vee (\neg c), (\neg c) \vee b \vee (\neg b) \vee (\neg c)\}$$

Apelul EliminaTautologii(γ) $\Rightarrow$ elimină clauza tautologie

$$(\neg c) \vee b \vee (\neg b) \vee (\neg c)$$

$$\gamma \leftarrow \{(\neg b) \vee c, (\neg c) \vee b \vee (\neg c)\}$$

Apelul EliminaDuplicate(γ) $\Rightarrow$ elimină duplicatul literalului $(\neg c)$ din

$$(\neg c) \vee b \vee (\neg c)$$

$$\gamma \leftarrow \{(\neg b) \vee c, (\neg c) \vee b\}.$$

Iterația 2: Nu există clauze unitare și nici literal puri.

Apelul alege(λ literal) $\Rightarrow \lambda = b$

$$\gamma \leftarrow REZ_b(\gamma) = \{c \vee (\neg c)\}$$

Apelul EliminaTautologii(γ) $\Rightarrow$ elimină clauza tautologie $c \vee (\neg c)$

$$\gamma \leftarrow \emptyset$$

Apelul EliminaDuplicate(γ) $\Rightarrow$ nu determină modificări.

Iterația 3: $\gamma = \emptyset \Rightarrow$ write ('validabilă'), $sw \leftarrow true \Rightarrow$ stop.